Lecture Notes: Self-supervised learning

Course: 2110573 Pattern Recognition

2026-03-05
version 0.1

1 Introduction to Self-Supervised Learning

This lecture introduces the concept of self-supervised learning (SSL), placing it in the context of other machine learning paradigms like unsupervised and semi-supervised learning. The main goal of SSL is to leverage large amounts of unlabeled data to learn meaningful representations that can then be used for downstream tasks with minimal labeled data.

1.1 Recap of Deep Learning Concepts

Before diving into self-supervised learning, let's recap some fundamental building blocks of deep learning:

- **Layers**:
  - Fully connected, CNN, RNN/GRU
  - Attention
  - Normalization Layers (Batch/Layer/Group/Instance Norms)
  - Dropout, Softmax
  - Residual connections
- **Architectures**:
  - ResNet, Xception
  - Transformer

1.2 Flavors of Supervision

Machine learning can be categorized based on the type and amount of supervision provided during training.

- **Supervised Learning**: All data points are labeled. The goal is to learn a mapping from inputs to outputs.
- **Semi-supervised Learning**: A small amount of labeled data is available along with a large amount of unlabeled data.
- **Unsupervised Learning**: No labels are provided. The goal is to find hidden patterns or structures in the data. This includes tasks like clustering and dimensionality reduction.
- **Self-supervised Learning**: A subset of unsupervised learning where the supervision signal is generated from the data itself. It's often used for representation learning. A surrogate task is created using pseudo labels.
- **Reinforcement Learning**: An agent learns to make decisions by taking actions in an environment to maximize a cumulative reward.

2 Unsupervised and Semi-Supervised Learning

2.1 What is Unsupervised Learning?

Unsupervised learning aims to utilize unlabeled data. A common misconception is that an unsupervised model can, for instance, identify a cat in an image without any form of labeling. This is not the case. The core idea is to learn meaningful representations from the data that can then be useful for other tasks.

2.2 Semi-supervised Learning

Semi-supervised learning bridges the gap between supervised and unsupervised learning by using both labeled and unlabeled data. The challenge is to make use of the vast amount of unlabeled data to improve upon a model trained only on the small labeled set.

2.2.1 Idea 1: Bootstrapping with Self-training

A simple yet effective technique for semi-supervised learning.

1. Train a classifier on the available labeled data.
2. Use this classifier to predict labels for the unlabeled data, creating "pseudo labels".
3. Filter the pseudo-labeled data, keeping only the predictions with high confidence scores.
4. Combine the original labeled data with the high-confidence pseudo-labeled data and retrain the classifier.

Caveats:

- The model's confidence scores can be misleading, though their ranking is often reliable.
- The model might reinforce its own mistakes by learning from incorrect pseudo labels.
- Performance can degrade if the unlabeled data comes from a very different distribution than the labeled data.

2.2.2 Idea 2: Tri-training

Tri-training improves upon self-training by using multiple models.

1. Train three different models on different subsets of the labeled data.
2. For each unlabeled data point, if two of the models agree on a label, that label is assigned to the data point.
3. This newly labeled data point is then used to train the third model.

A variation, **tri-training with disagreement**, suggests that it's more useful to use data points where two models agree and the third disagrees, as these are likely to be more informative than "easy" data points where all models agree.

2.2.3 Notes on Pseudo Labels

Modern techniques often prefer "soft" labels (probability distributions over classes) over "hard" labels (a single class). This is analogous to using Gaussian Mixture Models (GMM) instead of k-means for clustering. Pseudo-labeling has been shown to be very effective in practice and, with a strong starting model, can sometimes even outperform models trained on noisy human-provided labels.

3 Representation Learning

The goal of representation learning is to find a function f that maps high-dimensional input data (like an image) to a lower-dimensional representation (a vector of numbers) that is "useful".

3.1 What makes a representation useful?

An ideal representation should capture the essential semantic features of the input. For example, for an image of a cat, a good representation might encode features like "animal species" and "color". Such a representation would make downstream tasks, like animal classification, much easier—perhaps even solvable with a simple linear classifier. An un-useful representation might capture trivial features like the value of the top-left pixel or the standard deviation of pixel colors.

The challenge is to learn the most useful function f from unlabeled data. The definition of "usefulness" is subjective but is often related to performance on tasks that humans find natural.

3.2 Classical Approaches: PCA

Principal Component Analysis (PCA) is a linear function that can be used for representation learning. For example, in facial recognition, PCA is used to find "Eigenfaces", which form a basis for representing face images. However, PCA has limitations:

- It's a linear transformation, while many interesting attributes (like identity or emotion) are highly non-linear.
- It requires data to be well-aligned and centered.
- It is sensitive to variations in illumination.

3.3 Learning Representations with Deep Learning

Deep neural networks (DNNs) are powerful tools for learning complex, non-linear representations. The key is to use clever tricks to train these networks without manual labels.

4 Self-Supervised Learning

Self-supervised learning (SSL) is a paradigm for training deep learning models on unlabeled data. It works by creating a pretext, or surrogate, task where the supervision signal comes from the data itself.

As Yann LeCun puts it, SSL is about "filling in the blanks." The system learns to predict a part of its input from other parts of the input.

- Predict the future from the past.
- Predict a masked portion from the visible portion.
- Predict any occluded part from the available parts.

4.1 Autoencoders

An autoencoder is a type of neural network used for unsupervised representation learning. "Auto" means "self," as it learns to reconstruct its own input.

- An **encoder** network maps the input X to a low-dimensional representation z, often called the "code" or "bottleneck".
- A **decoder** network reconstructs the input X' from the code z.

The network is trained by minimizing a loss function that measures the difference between the input and the reconstructed output, such as the L2 loss:

$$\mathcal{L} = ||X - X'||^2 \tag{1}$$

For the network to learn a meaningful representation, the dimension of the code z must be smaller than the dimension of the input X. This forces the encoder to learn a compressed representation that captures the most important attributes of the data.

4.1.1 Denoising Autoencoders

To make the learned representations more robust, we can use a **denoising autoencoder**. The process is as follows:

1. Corrupt the input X to get a noisy version X'' (e.g., by adding Gaussian noise or blurring the image).
2. Feed the corrupted input X'' to the autoencoder.
3. Train the network to reconstruct the original, clean input X. The loss is $||X - X'||^2$.

This forces the model to learn to separate the signal from the noise, leading to more robust features. The choice of noise is crucial: it should be significant enough to force the model to learn important features but not so much that it fundamentally changes the data.

4.2 Consistency Training / Teacher-Student Models

This is a powerful SSL technique where a "teacher" model and one or more "student" models are used.

- A main **teacher** model makes predictions on unlabeled data.
- **Student** models are shown a limited or augmented "view" of the data and are trained to match the teacher's predictions.

This forces the students to learn representations that are consistent across different views of the data. Good data augmentation is key to this process. Examples of augmentation include back-translation for text or RandAugment for images.

4.3 Pre-trained Transformer Models

The transformer architecture, particularly when pre-trained using self-supervised methods, has revolutionized NLP and is now making a big impact in computer vision.

- **Encoder-only (Autoencoder models)**: e.g., BERT. Trained to predict masked words. Good for tasks like sequence classification.
- **Decoder-only (Autoregressive models)**: e.g., GPT. Trained to predict the next word in a sequence. Good for text generation.
- **Encoder-Decoder (Seq2Seq models)**: e.g., BART, T5. Trained on tasks like text summarization or machine translation.

The famous **BERT (Bidirectional Encoder Representations from Transformers)** uses two self-supervised tasks:

1. **Masked Language Model (MLM)**: Predict randomly masked words in a sentence.
2. **Next Sentence Prediction (NSP)**: Predict whether two sentences appeared consecutively in the original text.

5 Contrastive Learning

Contrastive learning is a popular and powerful self-supervised learning technique. The core idea is to learn a representation where similar (positive) samples are pulled closer together and dissimilar (negative) samples are pushed further apart in the embedding space.

5.1 Consistency vs. Contrastive Training

- **Consistency training** focuses on making the representations of different views (augmentations) of the same sample similar.
- **Contrastive training** adds the element of pushing away representations of different samples.

These two are often used together.

5.2 Triplet Loss

One of the earliest contrastive losses, commonly used in face verification. For a given "anchor" sample, a "positive" sample (from the same class) and a "negative" sample (from a different class) are chosen. The loss function aims to ensure that the distance between the anchor and the positive is smaller than the distance between the anchor and the negative by at least a certain margin α.

$$\mathcal{L} = \sum_{i=1}^{N} \left[||f(x_i^a) - f(x_i^p)||^2 - ||f(x_i^a) - f(x_i^n)||^2 + \alpha \right]_+ \tag{2}$$

where $[x]_+ = \max(0, x)$.

5.3 InfoNCE Loss

The InfoNCE (Noise Contrastive Estimation) loss is a form of contrastive loss that is widely used in modern SSL. Given a context vector c (e.g., the representation of an anchor) and a set of samples including one positive x and multiple negatives x', the loss is a categorical cross-entropy that tries to classify the positive sample correctly.

$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E}\left[\log \frac{f(x, c)}{\sum_{x'} f(x', c)} \right] \tag{3}$$

Here, $f(\cdot, \cdot)$ is a similarity function, often a cosine similarity scaled by a temperature parameter.

5.4 Key Details for Contrastive Learning

- **Large batch size**: A large batch provides more negative examples, which is crucial for learning good representations.
- **Temperature tuning**: The temperature parameter in the similarity function is a very important hyperparameter.
- **Hard negative mining**: Selecting challenging negative examples can improve performance.
- **Data augmentation**: Strong data augmentation is essential for creating effective positive pairs.

5.5 Contrastive learning in NLP example: SimCSE (Simple Contrastive Sentence Embeddings)

A key goal in NLP is to obtain high-quality **sentence embeddings**—fixed-size vector representations of sentences that capture their semantic meaning. A simple approach like averaging the token embeddings from a pre-trained model like BERT often yields poor results. SimCSE is a simple but powerful contrastive framework for learning excellent sentence embeddings.

5.5.1 Unsupervised SimCSE

The core insight of unsupervised SimCSE is to use standard **dropout** as minimal data augmentation.

1. A sentence is fed through a pre-trained Transformer encoder (like BERT) twice.
2. Because dropout is active during training, it acts as a stochastic process, producing two slightly different embeddings for the same sentence. These two embeddings form a **positive pair**.
3. **Negative pairs** are simply other sentences within the same training batch.
4. The model is trained using the InfoNCE loss to pull the positive pairs together and push apart the negative pairs.

This method is surprisingly effective and demonstrates that dropout noise is a sufficient form of augmentation to learn robust sentence representations.

5.5.2 Supervised SimCSE

When labeled data is available, SimCSE can leverage it to create more meaningful positive pairs.

- Using datasets from Natural Language Inference (NLI), which contain pairs of sentences (a premise and a hypothesis) labeled as entailment, neutral, or contradiction.
- A **premise** and its corresponding **entailment** are treated as a positive pair (e.g., "A man is running" and "A person is exercising").
- The **contradiction** sentences from the same premise are used as **hard negatives**, which are more challenging and informative than random negatives.

This supervised approach further improves the quality of the learned sentence embeddings by incorporating explicit semantic similarity signals.

5.6 Modern SSL Vision Models

- **MoCo (Momentum Contrast)**: Uses a queue to maintain a large and consistent set of negative samples, decoupling the batch size from the number of negatives.
- **SimCLR**: Uses a very large batch size and a simple contrastive loss.
- **DINO / Dinov2**: A self-distillation framework with no labels, using a teacher-student setup. It has shown remarkable properties, like unsupervised object segmentation.
- **Masked Autoencoders (MAE)**: Inspired by BERT, it masks a large portion of image patches and learns to reconstruct the missing patches. It is one of the most effective SSL methods for vision.
- **CLIP (Contrastive Language-Image Pre-training)**: Learns visual representations from natural language supervision by training on a massive dataset of image-text pairs from the internet. It can perform zero-shot classification with high accuracy.

6 World Models

A World Model is a generative neural network trained to learn a compressed spatial and temporal representation of an environment, essentially creating an internal "dream" or simulation. This concept, popularized by the 2018 paper by Ha & Schmidhuber, allows an agent to understand its environment's dynamics and predict future outcomes of its actions. This predictive capability enables the agent to learn and plan within its simulated world, which is significantly more sample-efficient than learning through direct trial-and-error in the real environment.

6.1 Core Architecture

The original World Model architecture consists of three main components that are trained separately:

- **Vision Model (V)**: This is a Variational Autoencoder (VAE) that processes high-dimensional observations (e.g., game pixels) and compresses them into a low-dimensional latent vector, z. This vector captures the essential features of the current state while filtering out noise. The VAE is trained in an unsupervised manner to reconstruct the input observation.
- **Memory Model (M)**: This is a Recurrent Neural Network (RNN) that models the temporal dynamics of the environment within the latent space. It takes the current latent vector z_t and the action a_t taken by the agent to predict the next latent vector, z_{t+1}. This component acts as the predictive brain of the agent, allowing it to "see" into the future.
- **Controller (C)**: This is a very simple and compact policy network (often a linear model) that decides which action to take. Its key feature is that it operates entirely on the latent representations. It takes the current latent state z_t from the Vision model and the hidden state h_t of the Memory RNN as input to select an action a_t.

6.2 Learning Inside the Dream

The power of the World Model comes from separating the learning of the world's dynamics from the learning of a task-specific policy.

1. The VAE and RNN are first trained on a collection of random experiences to learn a general model of the environment, without any knowledge of rewards.
2. The small Controller is then trained to solve a specific task (e.g., driving a car, winning a game). Crucially, this training can happen entirely within the "dream" generated by the Memory Model (M). The agent can simulate thousands of future scenarios in latent space without ever taking a step in the real world.
3. Because the controller is so small (only a few hundred parameters), it can be trained very efficiently using evolution strategies (like CMA-ES) to maximize the expected reward over the simulated trajectories.
4. The policy learned entirely within this simulated environment can then be transferred back to the actual environment to successfully perform the task.

This approach provides massive improvements in sample efficiency and allows agents to learn complex behaviors that would be impractical with traditional model-free reinforcement learning methods.

7 Transfer Learning

Transfer learning is the process of taking a model pre-trained on a large dataset (e.g., ImageNet) and adapting it for a new, often smaller, dataset. This pre-trained model is often called a "backbone".

7.1 The Process

1. **Pre-training**: A model is trained on a large-scale dataset (e.g., ImageNet for computer vision).
2. **Fine-tuning**: The pre-trained model is adapted for a new task.
   - The final classification layer of the pre-trained model is replaced with a new layer suitable for the new task.
   - The weights of the network are then fine-tuned. This can be done by:
     - **Freezing weights**: Training only the newly added layers. This is fast and effective if the new task is similar to the original task.
     - **Unfreezing all weights**: Training the entire network with a low learning rate. This can lead to better performance if the new dataset is large enough.

7.2 Benefits of Transfer Learning

- **Higher start**: The model starts with better performance than a randomly initialized model.
- **Higher slope**: The model converges faster.
- **Higher asymptote**: The model can achieve better final performance, especially with small datasets.

8 Summary

This lecture covered the progression from supervised to self-supervised learning.

- **Semi-supervised learning** uses a mix of labeled and unlabeled data, often by creating pseudo labels.
- **Representation learning** is about finding useful, compact representations of data.

- **Self-supervised learning** is a powerful paradigm for representation learning from unlabeled data by creating surrogate tasks. Key techniques include autoencoders, consistency training, and contrastive learning.
- These learned representations can then be effectively used for a wide range of downstream tasks through **transfer learning**.